

Design & Implementation of a Field-Oriented Control for a BLDC Motor on an FPGA

First Author^{#1}, Second Author^{*2}, Third Author^{#3}

[#]First-Third Department, First-Third University
Address Including Country Name

¹first.author@first-third.edu

³third.author@first-third.edu

Abstract—Field-oriented control (FOC) provides decoupled control of torque and flux, fast dynamic response and smooth torque over a wide speed range, which makes it attractive for brushless DC (BLDC) motor drives. This paper presents the design and implementation of a complete FOC system for a BLDC motor on a field-programmable gate array (FPGA). The design integrates cascaded position, velocity and d-q current loops; Clarke, Park, inverse Park and inverse Clarke transformations; a CORDIC-based sine/cosine generator; PID controllers with anti-windup; min-max space vector modulation; an initial rotor-position alignment routine; and encoder-based velocity estimation with a moving-average filter. Fixed-point scaling of the ADC, angle, current, encoder and PWM signals is derived in detail, and the data path is pipelined for deterministic timing. Simulation results verify the transform chain and scaling blocks, and hardware results are reported for velocity-loop and position-loop operation, including operation under an 80 N·m load.

Index Terms—BLDC motor, CORDIC, field-oriented control, FPGA, PID anti-windup, space vector modulation, vector control.

I. INTRODUCTION

Field-oriented control (FOC), also known as vector control, is preferred in relatively advanced applications despite its computational complexity [1], [2]. Unlike trapezoidal commutation, it can deliver high torque even at high speeds. Its advantages can be summarized as the transformation of the complex AC machine model into a simpler linear one, decoupled control of torque and flux, fast dynamic response, a high torque-to-current ratio at start-up, high efficiency and an extended speed range. FOC combines the strengths of trapezoidal and sinusoidal commutation: it offers the smoothness of sinusoidal commutation at low speed and remains readily applicable at high speed. In addition to the wider operating range, FOC yields lower total harmonic distortion (THD), an improved power factor, reduced switching losses and lower current ripple, qualities that satisfy general drive design constraints.

In FOC, the currents flowing through the stator—the stationary, outer part of the motor—are represented as two orthogonal components that can be treated as a vector. Realizing this requires the Clarke [3], Park [4], inverse Park and inverse Clarke transformations. The resulting voltage commands are passed to a space vector pulse-width

modulation (SVPWM) module, which shapes them so that the power stage (inverter) can supply each of the three motor phases with the required voltage and current at every instant.

A field-programmable gate array (FPGA) is a natural platform for such a controller because the transformation, trigonometric and PID stages can be pipelined and executed with deterministic timing. This paper describes the implementation of a complete FOC system for a BLDC motor on an FPGA. The design includes cascaded position, velocity and d-q current loops, a CORDIC-based sine/cosine generator, PID controllers with anti-windup, min-max space vector modulation, an initial rotor-position alignment routine, encoder-based velocity estimation with filtering, and the fixed-point scaling calculations that tie these blocks to the ADC, encoder and PWM hardware. Simulation and hardware results are presented, together with a comparison against commutation-based control and an existing controller, and the resulting FPGA resource utilization and power consumption.

The remainder of this paper is organized as follows. Section II presents the FOC architecture. Sections III–VIII describes the reference-frame transformations, the CORDIC module, the PID controller with anti-windup, space vector modulation, initial position alignment and velocity estimation. Section IX derives the scaling calculations, Section X reports the results, and Section XI concludes the paper.

II. FOC ARCHITECTURE

A. Functional Modules

The FOC system is composed of the following modules: position loop; velocity loop; CORDIC; Clarke, Park, inverse Clarke and inverse Park transforms; d-axis and q-axis current control loops; space vector modulation; incremental encoder interface; EMA current loop; initial-angle algorithm; mechanical-to-electrical angle conversion; voltage-to-PWM conversion; and velocity filter.

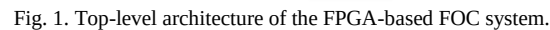
B. Top-Level Architecture

Fig. 1 shows the top-level architecture. Phase currents are measured by current sensors and the rotor position by an incremental encoder. The current sensor produces an analog output, which must be digitized before it can be processed in the digital domain; because the sensor has a bias of 2.5 V at 0 A, this offset is removed before further processing. The

waveforms are converted to duty cycles by adding a bias, and six PWM signals are generated for the MOSFET driver. Until the initial alignment is complete, a multiplexer selects the fixed values produced by the initial-angle routine instead of the current-loop outputs (Section VII).

C. Pipelined Architecture

To obtain deterministic latency, the data path is pipelined as shown in Fig. 2. Register stages, each annotated 10 μ s in the figure, separate the position loop, the velocity loop, the Clarke and Park transform, the CORDIC, the d and q current loops, the inverse Park and Clarke transform, and the space vector modulation block. The principal design parameters are summarized in Table I.



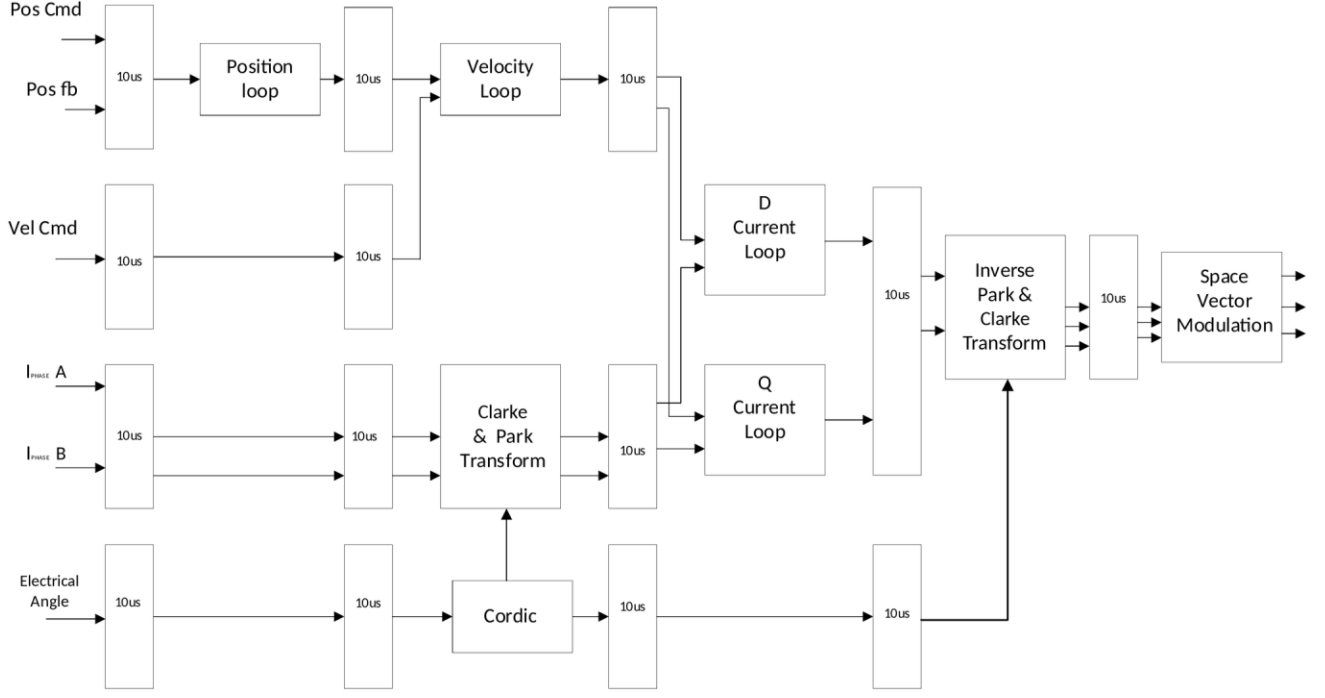


Fig. 2. Pipelined architecture of the FOC datapath (each register stage is annotated 10 μ s).

TABLE I
KEY DESIGN PARAMETERS

Parameter	Value
PWM switching frequency	10 kHz
PWM dead band	1 μ s
PWM full-scale count (100 % duty)	6400 (bias 3200)
Pipeline stage interval	10 μ s
ADC	8 channels, 14-bit, ± 10 V
ADC resolution	1.22 mV
Current sensor	20 mV/A, 2.5 V bias at 0 A
Incremental encoder	512 PPR, $\times 4 = 2048$ CPR
Alignment command / duration	$I_d = 10$ A, $I_q = 0$ A / 200 ms
Velocity window T_{duration}	10 μ s

III. REFERENCE-FRAME TRANSFORMATIONS

The phase currents of the motor depend on the rotor position, which increases the complexity of the motor equations. If the reference frame is moved from the stationary frame to one that rotates with the motor, the equations are considerably simplified [5].

A. Clarke Transform

The transformation from the three-phase (a, b, c) frame of voltages or currents to the two-axis stationary frame (α , β) is called the Clarke or $\alpha\beta$ transformation. After it, the vectors are described by only two orthogonal components (α , β). The a-axis and the α -axis are assumed to be aligned (Fig. 3(a)), and the amplitude-invariant form is

$$\alpha = \frac{2}{3} \left(a - \frac{1}{2}b - \frac{1}{2}c \right) \quad (1)$$

$$\beta = \frac{1}{\sqrt{3}} (b - c) \quad (2)$$

The resulting system is still time-varying. For a balanced three-phase system, $a + b + c = 0$, so that (1) and (2) simplify to

$$\alpha = a \quad (3)$$

$$\beta = \frac{a+2b}{\sqrt{3}} \quad (4)$$

where a and b are the phase currents A and B.

B. Park Transform

The transformation from the stationary frame (α , β) to the rotating reference frame (d, q) is called the Park transformation:

$$d = \alpha \cos \theta + \beta \sin \theta \quad (5)$$

$$q = -\alpha \sin \theta + \beta \cos \theta \quad (6)$$

This projection transforms the two-phase orthogonal system produced by the Clarke transformation into a rotating reference frame, in which the quantities are time-invariant. The only requirement is that the d-axis be aligned with the rotor flux (Fig. 3(b)). Flux and torque become the components of the new vector, and θ in (5) and (6) is the rotor flux position, which is either measured or estimated. Knowledge of the rotor flux position is central to the correct performance of FOC.

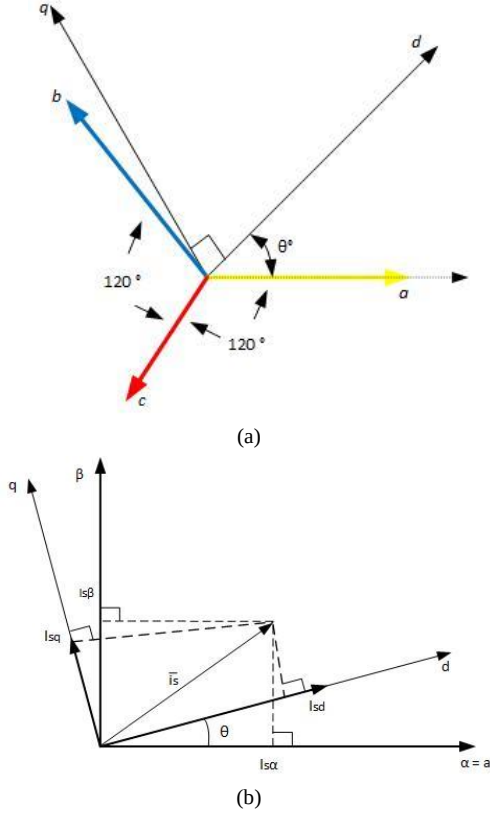


Fig. 3. Reference-frame transformations: (a) abc to $\alpha\beta$ (Clarke); (b) $\alpha\beta$ to dq (Park).

C. Inverse Park Transform

The transformation from the rotating (d, q) frame back to the stationary (α , β) frame is called the inverse Park transformation:

$$\alpha = d \cos \theta - q \sin \theta \quad (7)$$

$$\beta = d \sin \theta + q \cos \theta \quad (8)$$

D. Inverse Clarke Transform

The final step prepares the signals for the inverter hardware. The inverse Clarke transformation takes the two-axis stationary frame (α , β) and projects it back onto the physical three-phase axes, which are separated by 120° . The resulting three-phase voltage commands are then typically fed to an SVPWM algorithm, which drives the inverter transistors that power the motor windings:

$$a = \alpha \quad (9)$$

$$b = -\frac{1}{2}\alpha + \frac{\sqrt{3}}{2}\beta \quad (10)$$

$$c = -\frac{1}{2}\alpha - \frac{\sqrt{3}}{2}\beta \quad (11)$$

IV. CORDIC-BASED SINE/COSINE GENERATION

CORDIC (Coordinate Rotation Digital Computer) is an efficient iterative algorithm that generates trigonometric functions such as sine and cosine using only additions, subtractions, bit shifts and table lookups [6]. Because it requires neither multipliers nor dividers, CORDIC is widely

used in digital signal processors, microcontrollers and FPGA-based motor control systems [7].

The principle of the algorithm is vector rotation. Starting from an initial vector, a series of small predefined rotations is performed to reach the desired angle. In each iteration the direction of rotation is determined by the remaining angle error, and the vector components are updated continuously. After several iterations the final x and y components correspond to the cosine and sine of the target angle, respectively.

In an FOC system, CORDIC generates the sine and cosine values required by the Park and inverse Park transformations. Its high accuracy, low computational cost and hardware-friendly structure make it well suited to real-time motor control. Fig. 4 shows the simulation of the sine/cosine module. The angle input uses the sfix32_En16 format (16 fractional bits), so a full electrical revolution of 2π corresponds to $2\pi \times 2^{16} \approx 411\,775$ LSBs; the value 405 731 marked by the cursor in Fig. 4 corresponds to 6.19 rad.

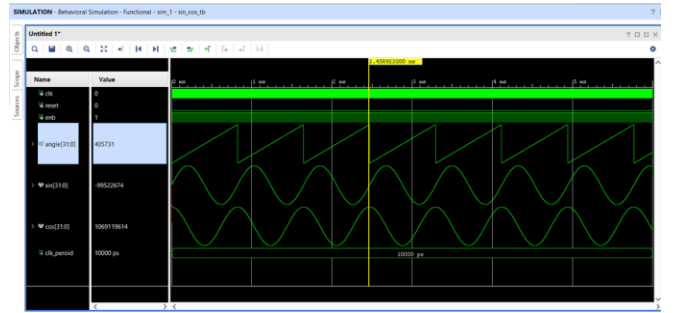


Fig. 4. Simulation of the sine/cosine (CORDIC) module.

In Fig. 5(a), an electrical angle ramping from 0 to 2π rad (6.28) is applied to the CORDIC module and the sine and cosine waveforms are obtained. In the expanded view of Fig. 5(b) the fundamental period is 1 s, and the cosine and sine peaks occur at 3.70 s and 3.95 s, respectively. The 0.25 s separation corresponds to a 90° phase difference, with the cosine leading the sine.

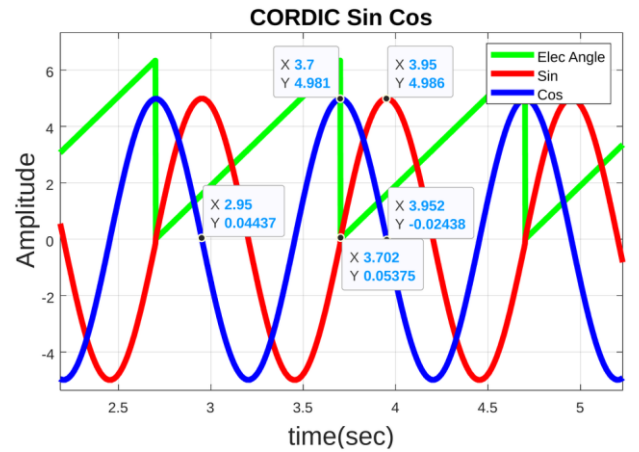


Fig. 5. MATLAB waveforms of the CORDIC sine/cosine module: electrical angle, sine and cosine

V. PID CONTROLLER WITH ANTI-WINDUP

In a PID controller the integral term accumulates the error over time to eliminate steady-state error and improve control accuracy [8]. In practical systems, however, the controller output is limited by actuator or inverter constraints. When the output reaches these limits the controller saturates, while the integral term may continue to accumulate error. This condition, known as integral windup, can cause excessive overshoot, oscillation and slow recovery when the controller returns to its normal operating range [9].

In the implemented controller (Fig. 6) the integrator is switched off when both of the following conditions are true simultaneously:

- the PID output before and after the saturation block is not equal;
- the sign of the error and the sign of the PID output are equal.

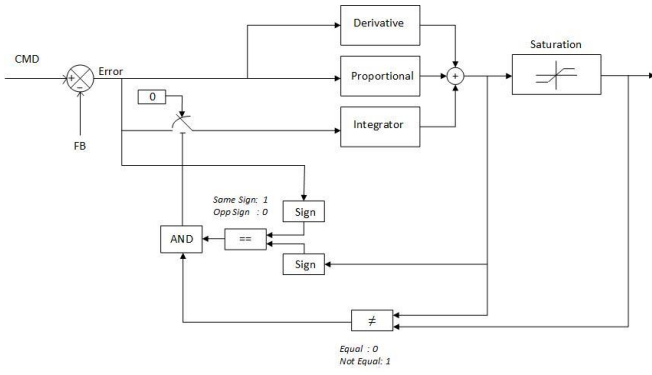


Fig. 6. Block diagram of the PID controller with anti-windup.

Anti-windup logic is widely used in the speed and current control loops of FOC applications. It allows the controller to recover quickly from saturation, reduces overshoot and ensures smooth motor operation during sudden speed changes or load disturbances, thereby improving the efficiency, accuracy and reliability of the motor control system. Fig. 7 shows the MATLAB response of the cascaded controller with $K_{p1} = 6$, $K_{p2} = 8$, $K_{i1} = 20$, $K_{i2} = 15$ and $K_{d1} = K_{d2} = 0$. The position and velocity anti-windup traces are non-zero only around the command transitions, when the controller saturates.

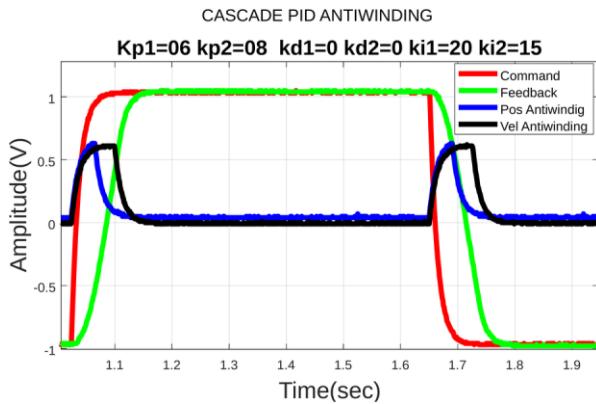


Fig. 7. MATLAB response of the cascaded PID controller with anti-windup.

VI. SPACE VECTOR MODULATION

Space vector pulse-width modulation (SVPWM) is an advanced PWM technique widely used in FOC. Compared with conventional sinusoidal PWM, it provides better utilization of the DC-bus voltage, lower harmonic distortion and improved motor performance [10]. It is commonly used in permanent magnet synchronous motor (PMSM) and induction motor drives owing to its high efficiency and smooth torque characteristics.

The min-max method is a simplified implementation of SVPWM with reduced computational complexity [11]. Instead of determining the inverter switching sector and calculating switching times, it directly uses the maximum and minimum values of the three-phase reference voltages A, B and C to generate the PWM signals. An offset voltage shifts the three reference waveforms so that they remain within the available DC-bus voltage range. As a result, the inverter can generate a larger fundamental output voltage without over-modulation, which improves DC-bus utilization by approximately 15 % compared with sinusoidal PWM. The modified references A_{SVM} , B_{SVM} and C_{SVM} are obtained as

$$x = \max(A, B, C) \quad (12)$$

$$y = \min(A, B, C) \quad (13)$$

$$z = \frac{x+y}{2} \quad (14)$$

$$A_{SVM} = A - z \quad (15)$$

$$B_{SVM} = B - z \quad (16)$$

$$C_{SVM} = C - z \quad (17)$$

The inputs A, B and C are three-phase signals with 120° phase shift; A_{SVM} , B_{SVM} and C_{SVM} are the outputs of the space vector modulation block. Fig. 8 shows the simulated SVM waveforms.

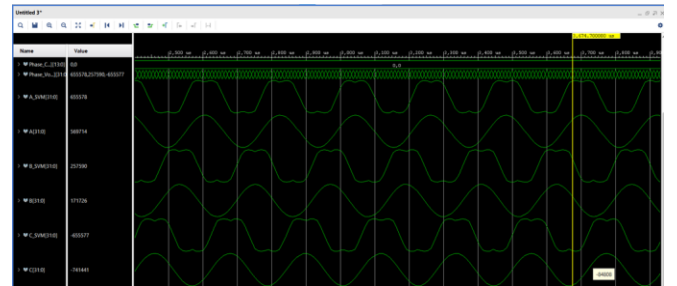


Fig. 8. Simulation waveforms of the space vector modulation outputs.

VII. INITIAL ROTOR-POSITION ALIGNMENT

Because the incremental encoder provides only relative position, the rotor must be aligned to a known reference at start-up. As shown in Fig. 9(a), a counter first runs for 200 ms (2 000 000 counts of a 10 MHz clock). During this interval I_d is set to its maximum value of 10 A, I_q is set to 0 A and the angle is held at 0°, so that the motor locks onto the d-axis, which defines the initial position. The “done” signal is 0 during this time. At the exact completion of the 200 ms the incremental counter is reset to zero, which makes this position the 0° reference, and the done signal is set to 1. From then on, the multiplexer of Fig. 9(b) passes the d and q currents

generated by the Clarke and Park transforms and the current loops instead of the fixed initial values.

For a motor with multiple pole pairs there is no unique d-axis: a four-pole motor, for example, has two d-axis locations per mechanical revolution. The rotor therefore locks to the d-axis location nearest to its initial position.

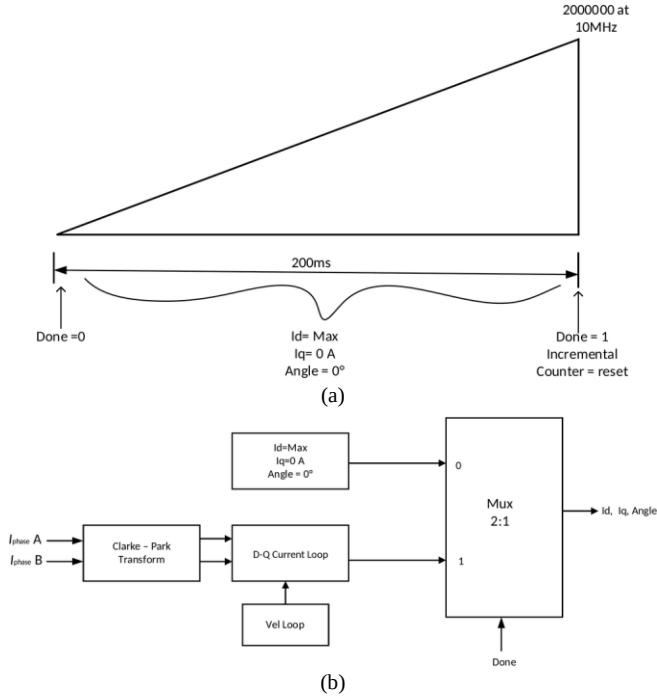


Fig. 9. Initial position alignment: (a) timing flow; (b) block diagram.

VIII. VELOCITY ESTIMATION FROM AN INCREMENTAL ENCODER

A. Quadrature Decoding

An incremental rotary encoder is typically mounted on the shaft of a mechanical system to provide motion or position information. It outputs two quadrature signals, which carry the relative change in position and the direction of rotation. A counter uses these signals to compute the change in position and keeps track of the most recent position value. As shown in Fig. 10, the direction is easily determined: if signal B lags signal A the rotation is clockwise, otherwise it is counter-clockwise. The angle θ is established from the encoder resolution.

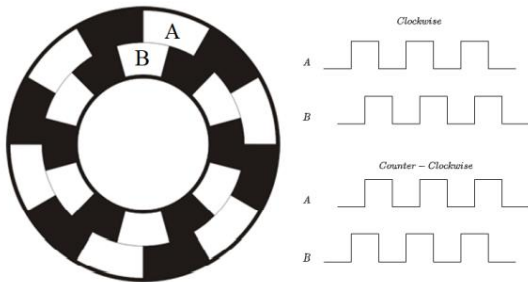


Fig. 10. Incremental encoder and its quadrature output waveforms.

B. Finite-Duration Time-Window Method

In the finite-duration time-window method, the pulses delivered by the incremental encoder are counted over a finite window and the motor velocity is estimated from the count. The count is converted to RPM by

$$RPM = \frac{N_p \times 60}{T_{duration} \times CPR} \quad (18)$$

where N_p is the number of pulses counted in the window, $T_{duration} = 10 \mu s$ is the window length and CPR is the number of counts per rotation. With an encoder of 512 pulses per rotation (PPR) and four-fold quadrature decoding, $CPR = 512 \times 4 = 2048$.

C. Velocity Filter

When the velocity is calculated from an incremental encoder, the FPGA samples the changing position at fixed time intervals ($\delta\theta/\delta t$). Because the encoder steps are discrete, this raw division produces a jagged signal with quantization noise, especially at low speeds where encoder pulses are sparse. If this noisy velocity is fed directly to the speed controller, the current loop overreacts, causing torque ripple, audible buzzing and poor control. A velocity filter is therefore used to smooth the digital jumps and extract the true speed of the mechanical shaft.

Common approaches are a digital low-pass filter or a moving-average filter; a moving-average filter, which continuously averages a sliding window of the last N raw speed calculations, is used in this design. Such a filter also introduces phase lag (time delay) into the control loop: if it is too aggressive, the speed controller reacts too late to real changes, causing overshoot or oscillation.

IX. SCALING CALCULATIONS

A. ADC Resolution

The FPGA board contains an 8-channel ADC with 14-bit resolution and an input range of $\pm 10 V$, i.e. a dynamic range of 20 V. The resolution for an n -bit converter is

$$\Delta = \frac{\text{Dynamic range}}{2^n} = \frac{20 V}{2^{14}} \approx 1.22 mV \quad (19)$$

To represent this in the sfixed32_En16 format the resolution is scaled by 2^{16} : $1.22 \times 10^{-3} \times 2^{16} \approx 80$, and an integer gain of 79 is used in the implementation (Fig. 11).

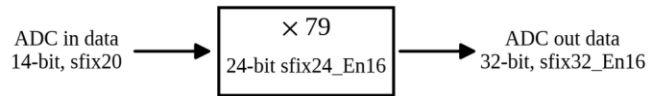


Fig. 11. ADC scaling block.

B. Mechanical to Electrical Angle

When the mechanical angle completes one 360° cycle, the electrical angle completes a number of cycles equal to the number of pole pairs P (Fig. 12):

$$\theta_e = P \theta_m \quad (20)$$

The encoder provides 2048 counts per mechanical revolution, so a mechanical angle of 0 to 2π corresponds to 0 to 2048. In the controller the angle $2\pi \approx 6.283$ is scaled by $819 \approx 2^{14}/20$ V (the ADC resolution factor), giving $6.283 \times 819 \approx 5145$. The scaling factor K therefore satisfies $2048 K = 5145$:

$$K = \frac{5145}{2048} \approx 2.512 \quad (21)$$

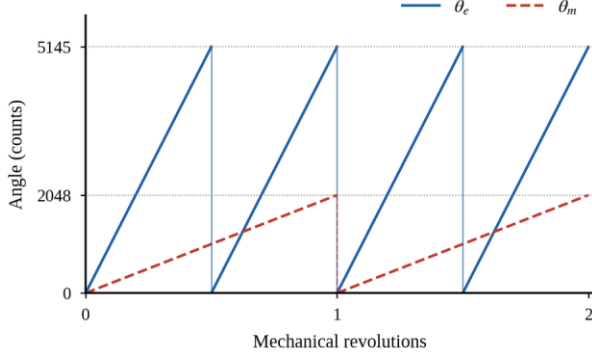


Fig. 12. Relationship between the mechanical angle θ_m and the electrical angle θ_e for a motor with two pole pairs.

Fig. 13 shows the MATLAB waveforms of the mechanical and electrical angles for $P = 2$. Fig. 14 shows the simulation of the conversion block for $P = 2$ for both directions of rotation (mechanical angle sweeping from 0 to 2π and from 2π to 0), and Fig. 15 shows the results for $P = 4, 6$ and 8 . In each case the electrical angle is the product of the mechanical angle and P wrapped at 2048: for example, for $P = 6$ a mechanical angle of 589 gives $589 \times 6 = 3534$, which wraps to an electrical angle of $3534 - 2048 = 1486$ (Fig. 15(b)).

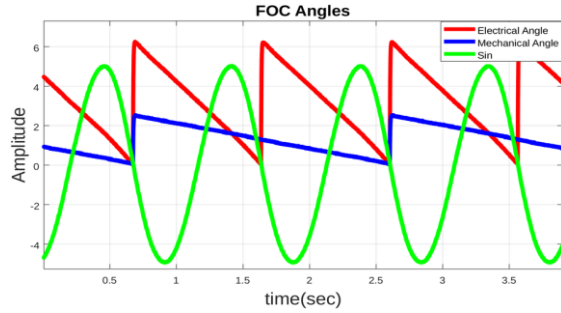
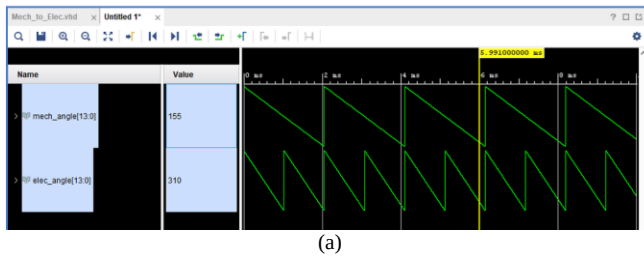
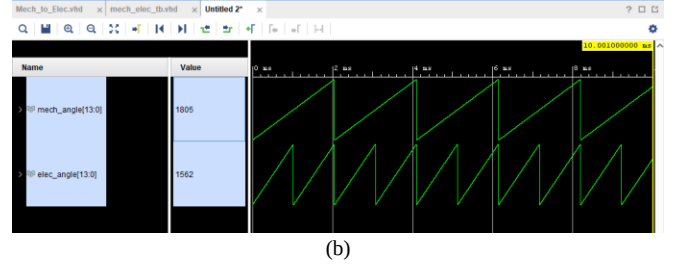


Fig. 13. MATLAB waveforms of the mechanical and electrical angles for two pole pairs.

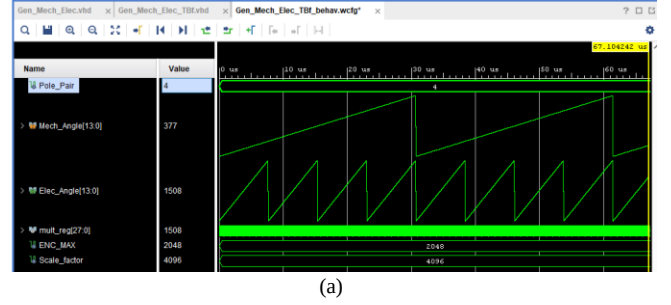


(a)

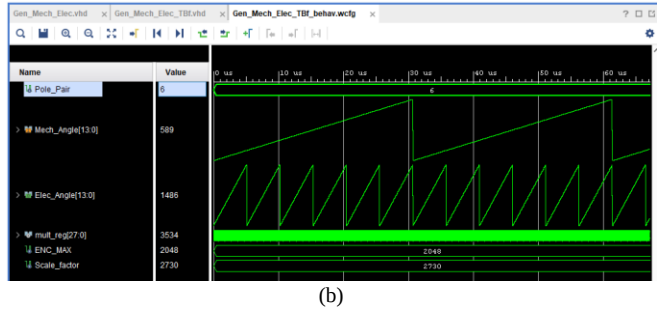


(b)

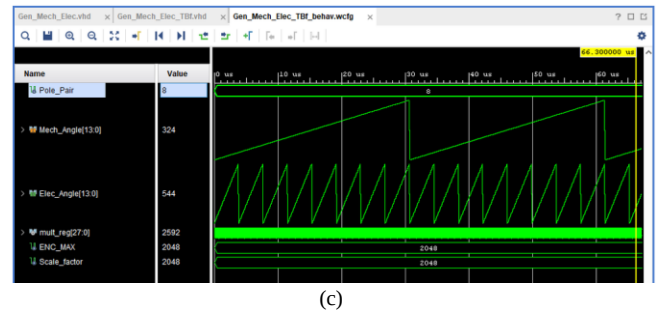
Fig. 14. Simulation of the mechanical-to-electrical angle conversion for $P = 2$ for the two directions of rotation.



(a)



(b)



(c)

Fig. 15. Simulation of the mechanical-to-electrical angle conversion for (a) $P = 4$, (b) $P = 6$ and (c) $P = 8$.

C. Incremental Encoder

The incremental encoder resolution is 2048 counts for a complete 360° rotation of the motor. Since 360° is represented as $2\pi \approx 6.28$, which corresponds to 5145 in the fixed-point scale, the encoder count is scaled by the same factor $K \approx 2.512$ obtained above.

D. Voltage to PWM

The SVM output ranges from $+65$ V to -65 V. To obtain the maximum duty within the linear modulation range, the limit

$$V_{lim} = \frac{2}{3} V_{dc} \cos\left(\frac{\pi}{6}\right) = \frac{V_{dc}}{\sqrt{3}} \quad (22)$$

is used instead of the DC-bus voltage V_{dc} directly, which gives a scaled full-scale value of ± 45.068 . With a full-scale count of 3200 and a PWM frequency of 10 kHz, the scaling factor is

$$K = \frac{3200}{45.068} \approx 71 \quad (23)$$

The SVM output therefore spans ± 3200 , and a bias of 3200 is added so that the duty count ranges from 0 to 6400, where 6400 corresponds to 100 % duty cycle. Fig. 17 shows the simulation of the FOC PWM with a dead band of 1 μ s in open loop.

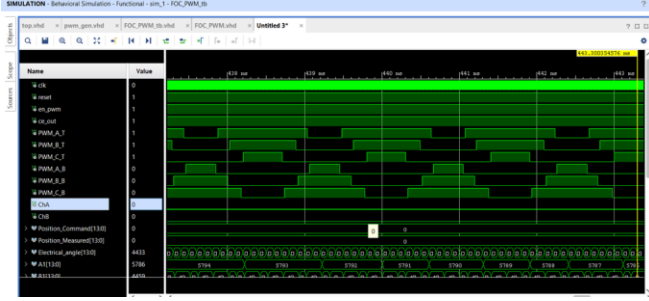


Fig. 17. Simulation of the FOC PWM outputs with a 1 μ s dead band (open loop).

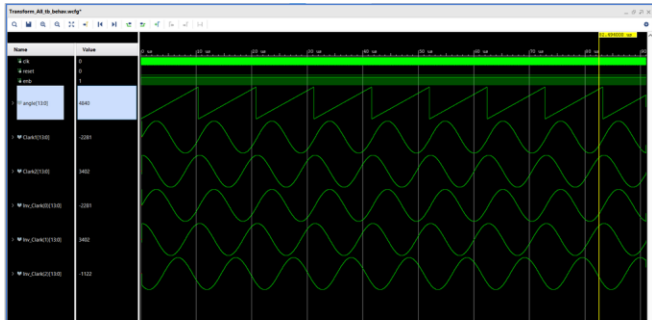
X. RESULTS AND DISCUSSION

A. Transform Simulation Results

Fig. 18(a) shows the simulation result of the Clarke transform followed by the Park transform for a rotating angle ramp, with the α , β and d , q signals. Fig. 18(b) shows the complete chain of Clarke, Park, inverse Park and inverse Clarke transforms. The reconstructed three-phase signals reproduce the original inputs (-2281 and 3402 at the cursor position), and the third phase (-1122) satisfies $a + b + c \approx 0$.



(a)



(b)

Fig. 18. Transform simulation: (a) Clarke and Park transforms; (b) complete chain (Clarke $\rightarrow$ Park $\rightarrow$ inverse Park $\rightarrow$ inverse Clarke).

B. Hardware Results

Velocity loop. Fig. 19 shows the velocity-loop response at motor level. The velocity feedback follows the step command closely and settles without visible overshoot.

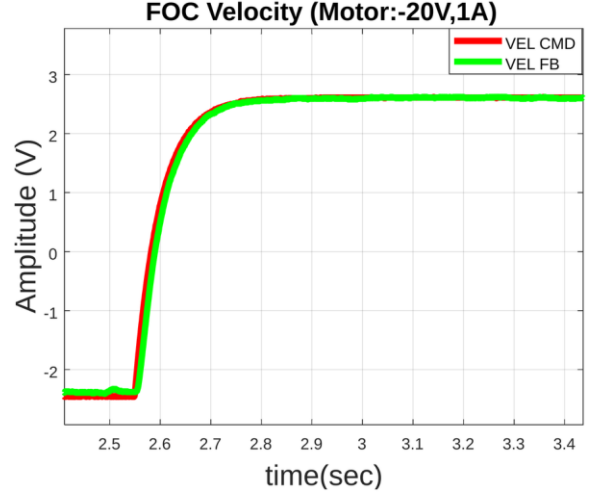
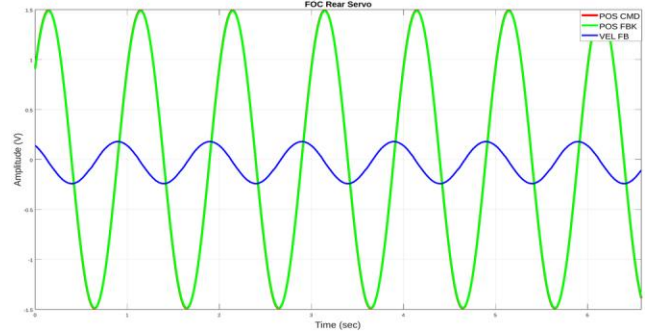
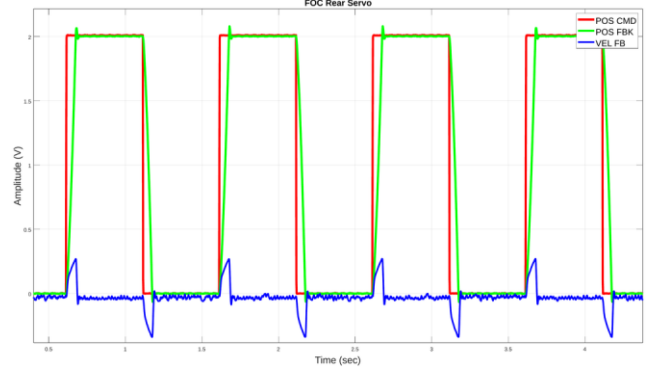


Fig. 19. Velocity-loop response at motor level (VEL CMD: command; VEL FB: feedback).

Fig. 20 shows the FOC position-loop response : sinusoidal tracking in Fig. 20(a), repeated 10° steps in Fig. 20(b) and an expanded view of a single step in Fig. 20(c). The position feedback (POS FBK) follows the command (POS CMD) in all cases.



(a)



(b)

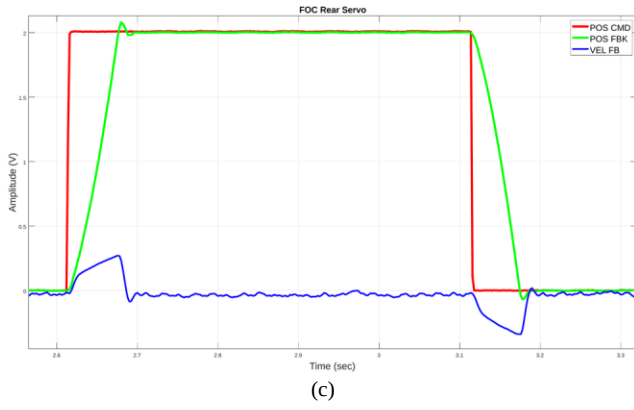


Fig. 20. FOC position-loop response: (a) sinusoidal command; (b) repeated 10° steps; (c) expanded view of one step.

Commutation position loop. Fig. 21 shows the response of the position command and position feedback for a step of +10° to −10° under commutation control.

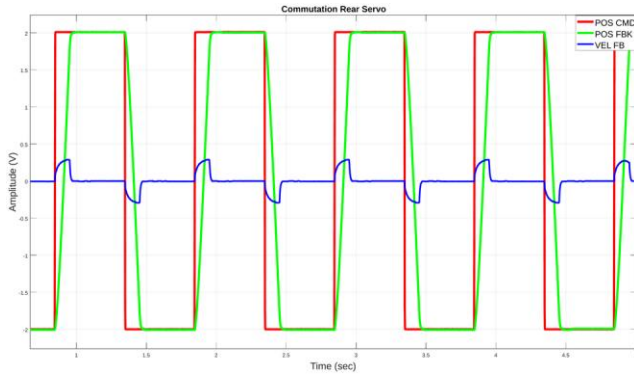


Fig. 21. Commutation position-loop response for +10° to −10°.

Operation under load. Fig. 22 shows FOC operation with load, displaying the position loop, the phase currents and the q-axis current loop for step commands and for a step followed by a sinusoidal command. Fig. 23 compares the phase currents at an 80 N·m load before and after proper tuning; the phase currents are clearly smoother after tuning.

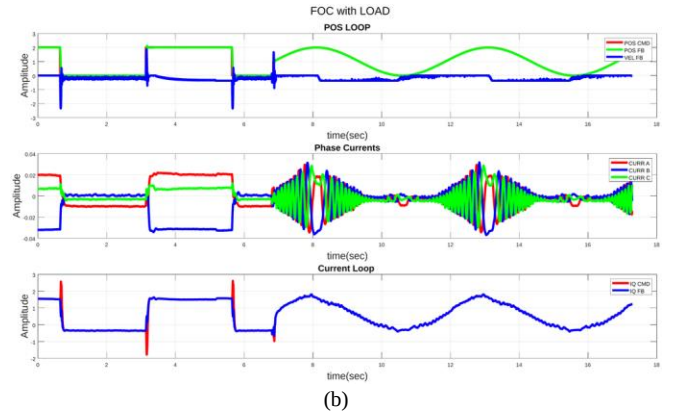
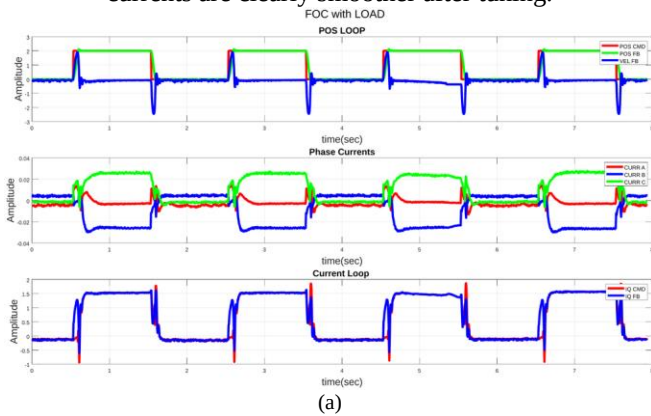


Fig. 22. FOC with load: position loop, phase currents and q-axis current loop for (a) step commands and (b) a step followed by a sinusoidal command.

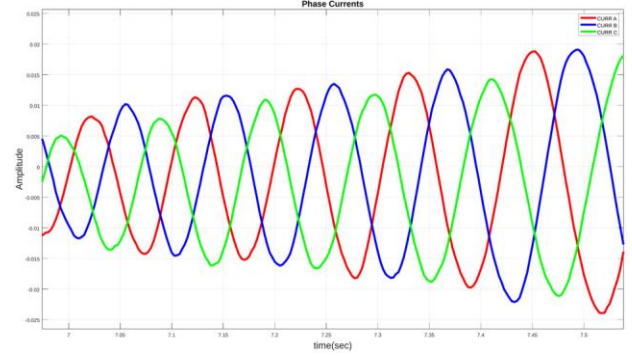


Fig. 23. Phase currents at 80 N·m load

C. Commutation and FOC Time-Parameter Comparison

Table II compares the time-domain step-response parameters of the FOC controller (10 kHz) and the commutation controller (10 kHz) for 10° and 20° position steps. Between the two 10 kHz controllers, FOC and commutation have similar rise times (47 ms versus 48 ms for 10°, and 83 ms versus 78 ms for 20°). FOC has the lower steady-state error (0.399 % versus 0.599 % for 10°, and 0.35 % versus 0.373 % for 20°), the shorter settling time for the 20° step (112 ms versus 136 ms) and no overshoot for the 20° step (versus 1.117 % for commutation), while the 10° overshoot is comparable (0.17 % versus 0.163 %).

TABLE II
TIME-DOMAIN COMPARISON OF THE FOC AND COMMUTATION CONTROLLERS

Controller	Step	Rise time (ms)	Speed (°/s)	Settling time (ms)	Steady-state error	Over-shoot
FOC (10 kHz)	10°	47	170	83	0.399 %	0.17 %
	20°	83	192	112	0.35 %	0
Commutation (10 kHz)	10°	48	168	84	0.599 %	0.163 %
	20°	78	204	136	0.373 %	1.117 %

D. FPGA Utilization

The synthesized design is mapped onto real hardware elements such as look-up tables, flip-flops and DSP slices; Table III lists the post-implementation utilization. The design uses 9824 of 63 400 LUTs (15.50 %), 7497 of 126 800 flip-

flops (5.91 %), 8 of 135 block RAMs (5.93 %) and 109 of 240 DSP slices (45.42 %). The DSP slices are the most heavily used resource.

TABLE III
POST-IMPLEMENTATION RESOURCE UTILIZATION OF THE FPGA

Resource	Used	Available	Utilization (%)
LUT	9824	63400	15.50
LUTRAM	243	19000	1.28
FF	7497	126800	5.91
BRAM	8	135	5.93
DSP	109	240	45.42
IO	44	210	20.95
BUFG	8	32	25.00
MMCM	1	6	16.67

E. Power Consumption

The total power required by the hardware elements generated during synthesis is 0.238 W, made up of 0.140 W (59 %) of dynamic power and 0.098 W (41 %) of static power, as broken down in Table IV. The PLL accounts for most of the dynamic power (0.089 W, 64 % of the dynamic total), followed by clocks (0.016 W) and signals (0.012 W).

TABLE IV
ON-CHIP POWER CONSUMPTION OF THE FPGA

Component	Power (W)	Share (%)
Dynamic	0.140	59
Clocks	0.016	11
Signals	0.012	9
Logic	0.009	6
BRAM	0.001	1
DSP	0.008	5
PLL	0.089	64
I/O	0.006	4
Static (PL)	0.098	41
Total on-chip	0.238	100

Percentages of the indented items are relative to the dynamic power.

XI. CONCLUSION

This paper presented the implementation of a complete field-oriented control system for a BLDC motor on an FPGA. The design was verified in simulation (transform chain, CORDIC, angle scaling and PWM with a 1 μ s dead band) and on hardware (velocity and position loops and operation under an 80 Nm load).

For 10° and 20° position steps, FOC at 10 kHz achieved rise times of 47 ms and 83 ms with steady-state errors of 0.399 % and 0.35 %. This is comparable to commutation at 10 kHz in rise time and better in steady-state error. After proper tuning, the phase currents at an 80 Nm load were markedly smoother. The design occupies 15.50 % of the LUTs, 5.91 % of the flip-flops and 45.42 % of the DSP slices of the target FPGA, with a total on-chip power of 0.238 W. Future work may address the remaining response-time gap to the reference controller.

REFERENCES

- [1] P. Vas, *Sensor less Vector and Direct Torque Control*. Oxford, U.K.: Oxford Univ. Press, 1998.
- [2] B. K. Bose, *Modern Power Electronics and AC Drives*. Upper Saddle River, NJ, USA: Prentice-Hall, 2002.
- [3] E. Clarke, *Circuit Analysis of A-C Power Systems*, vol. I. New York, NY, USA: Wiley, 1943.
- [4] R. H. Park, "Two-reaction theory of synchronous machines—Generalized method of analysis—Part I," *Trans. AIEE*, vol. 48, no. 3, pp. 716–727, Jul. 1929.
- [5] P. C. Krause, O. Wasynczuk, and S. D. Sudhoff, *Analysis of Electric Machinery and Drive Systems*, 2nd ed. Piscataway, NJ, USA: IEEE Press, 2002.
- [6] J. E. Volder, "The CORDIC trigonometric computing technique," *IRE Trans. Electron. Comput.*, vol. EC-8, no. 3, pp. 330–334, Sep. 1959.
- [7] R. Andraka, "A survey of CORDIC algorithms for FPGA based computers," in *Proc. ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, Monterey, CA, USA, 1998, pp. 191–200.
- [8] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Research Triangle Park, NC, USA: ISA, 1995.
- [9] C. Bohn and D. P. Atherton, "An analysis package comparing PID anti-windup strategies," *IEEE Control Syst. Mag.*, vol. 15, no. 2, pp. 34–40, Apr. 1995.
- [10] H. W. van der Broeck, H.-C. Skudelny, and G. V. Stanke, "Analysis and realization of a pulsewidth modulator based on voltage space vectors," *IEEE Trans. Ind. Appl.*, vol. 24, no. 1, pp. 142–150, Jan./Feb. 1988.
- [11] D.-W. Chung, J.-S. Kim, and S.-K. Sul, "Unified voltage modulation technique for real-time three-phase power conversion," *IEEE Trans. Ind. Appl.*, vol. 34, no. 2, pp. 374–380, Mar./Apr. 1998.